GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

TransferQueue

面向大模型后训练的高性能异步流式数据引擎

荣程浩 | 华为

张懿翔 | 华为

开源地址：https://github.com/Ascend/TransferQueue



CONTENTS

Part 01 后训练的数据新挑战

Part 02 TransferQueue 核心设计

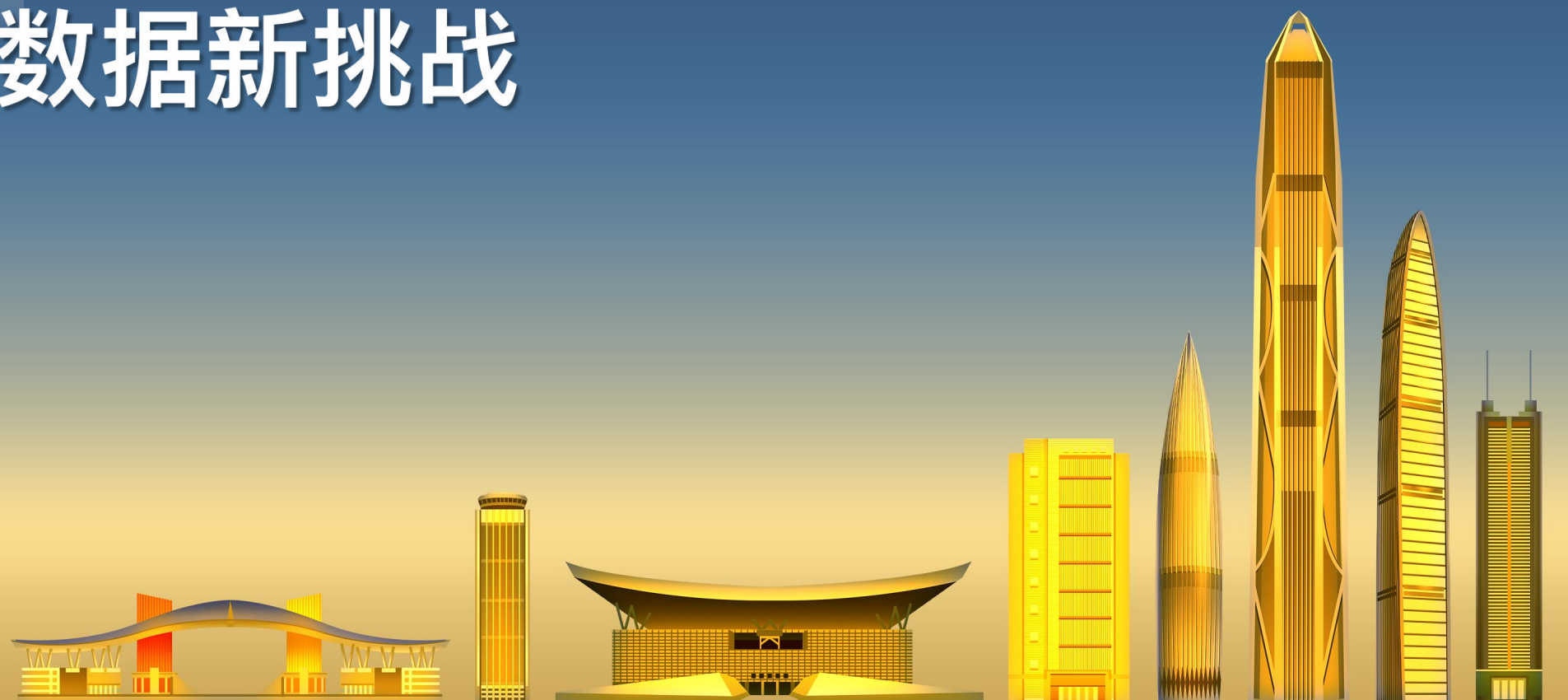
Part 03 性能收益

Part 04 开源项目与生态



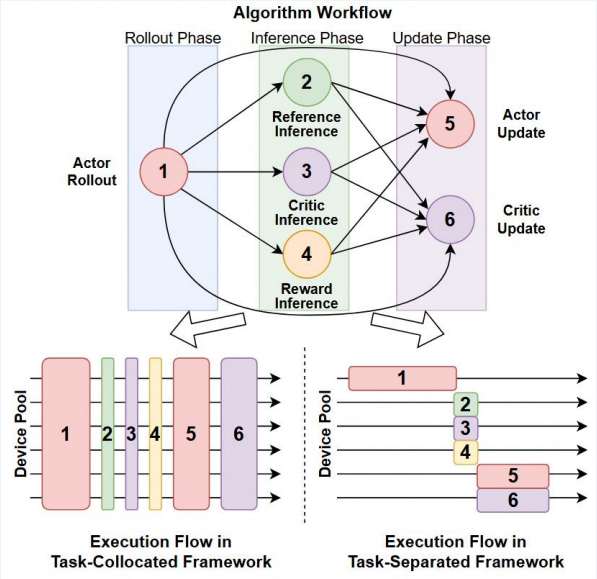
01

后训练的数据新挑战



后训练的三个新趋势

① 训推分离

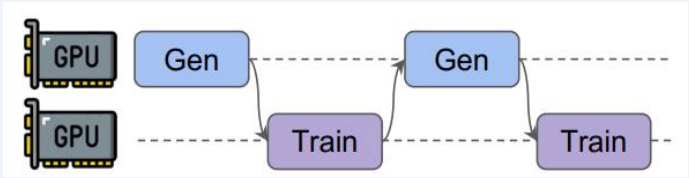


推理负责探索生成，训练负责参数更新；拆开后可**独立扩缩容**

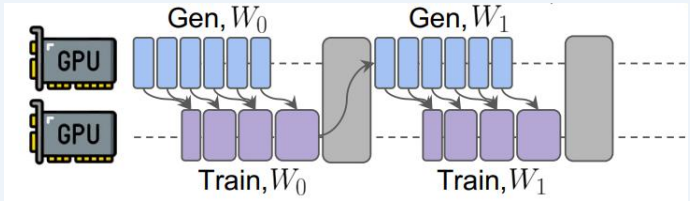
代价：原本进程内的数据流转，变成**跨进程、跨节点**的数据流动

② 从整批到流式

同步：被最慢的样本阻塞



异步：一个mini-batch就绪即进入下一阶段



智能体、多轮工具调用让完成时间呈**长尾分布**

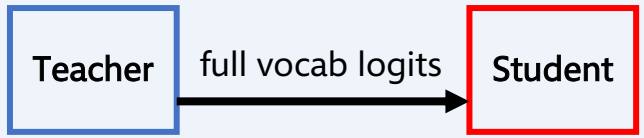
目标：在满足算法依赖的前提下，实现最小粒度的数据调度

③ 数据越来越重



Router Replay: expert IDs

1024 prompts × 16 samples × 32K tokens × 40 layers × 8 experts × 2 B ≈ 320 GB



OPD： logits（BF16）

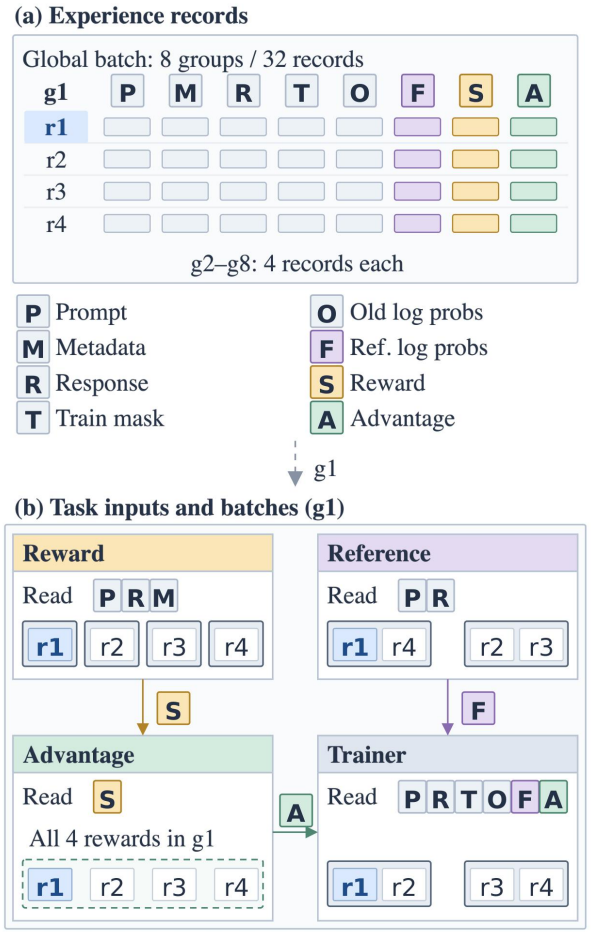
full-logits 256K ≈ 256 TB；top-2048 ≈ 2 TB

多模态：图片、视频、音频进入训练

新算法：更多中间状态与跨阶段字段

数据系统要回答：哪些数据已就绪？交给谁消费？如何减少不必要的搬运？

核心矛盾：一个 global batch 身兼三职



global batch 的三重职责

① 数据容器

所有字段打包成整条记录，随batch一起搬运

② complete barrier

整批完成后，下游任务才能开始

③ 生命周期管理

batch 结束，记录随之失效

现代工作负载的变化

字段变宽、变多

多模态与蒸馏让单条记录达数十MB；
不同任务只需部分字段。

就绪记录可以先被消费

同组 reward 就绪时间不同；长尾样本拖住整批

记录跨批次复用

回放、bounded reuse、异步 off-policy 训练

根因：数据面与控制面高度耦合

数据面：经验样本持续增长；

控制面：每个batch需要哪些数据，是临时的、任务私有的视图

—— 数据面与控制面应当解耦

实测：Batch绑定数据路径的代价

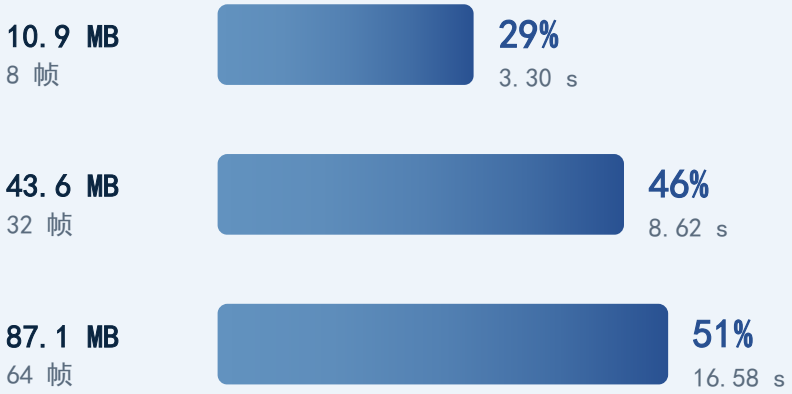
基线为 verl：只改变经验样本传输的方式，或改变负载大小，观察数据传输的时延变化

1.66×

集中式分发 → 分布式读取
820.3 s → 493.3 s / step

数据量越大，数据传输时延占比越高

视频 GRPO · Qwen3-VL-8B · 2×H200（8 卡 rollout + 8 卡训练）



−31% vs −6%

rollout 8→16 GPU：
rollout 任务时间减少31%，
训练侧阻塞时间仅减少6%

3.72 s

整个批次下，从首条记录就绪
到首个可用训练batch的等待时间

batch粒度被集中式传输，产生单点瓶颈

三项需求

把数据移到分布式存储

Worker 分布式读取存储，在 batch 不变的前提下，迭代时间 59.81 s → 39.10 s

但只解决了**单点瓶颈**这一个问题

仍然存在：

- 全量获取整条sample的所有经验样本
- batch 粒度的调度，长尾问题
- 消费即丢弃，难以跨迭代复用

↓ 推导出三项需求

R1 · 消费者直读

Consumer-directed delivery

只读取所需字段，直接从分布式存储读取，由任务自行组成执行批次

避免整条记录搬运和中心节点转发

R2 · 增量可用

Incremental availability

字段写完即可见；每个任务只等它需要的字段，而非整个 global batch

按 sample / 按 mini-batch 调度：

首个training batch 阻塞 3.72 s → 1.51 s

R3 · 有状态选择

Stateful selection

跟踪每条样本对每个任务是否仍可被消费，选取时更新消费状态；无任务再需要时才回收

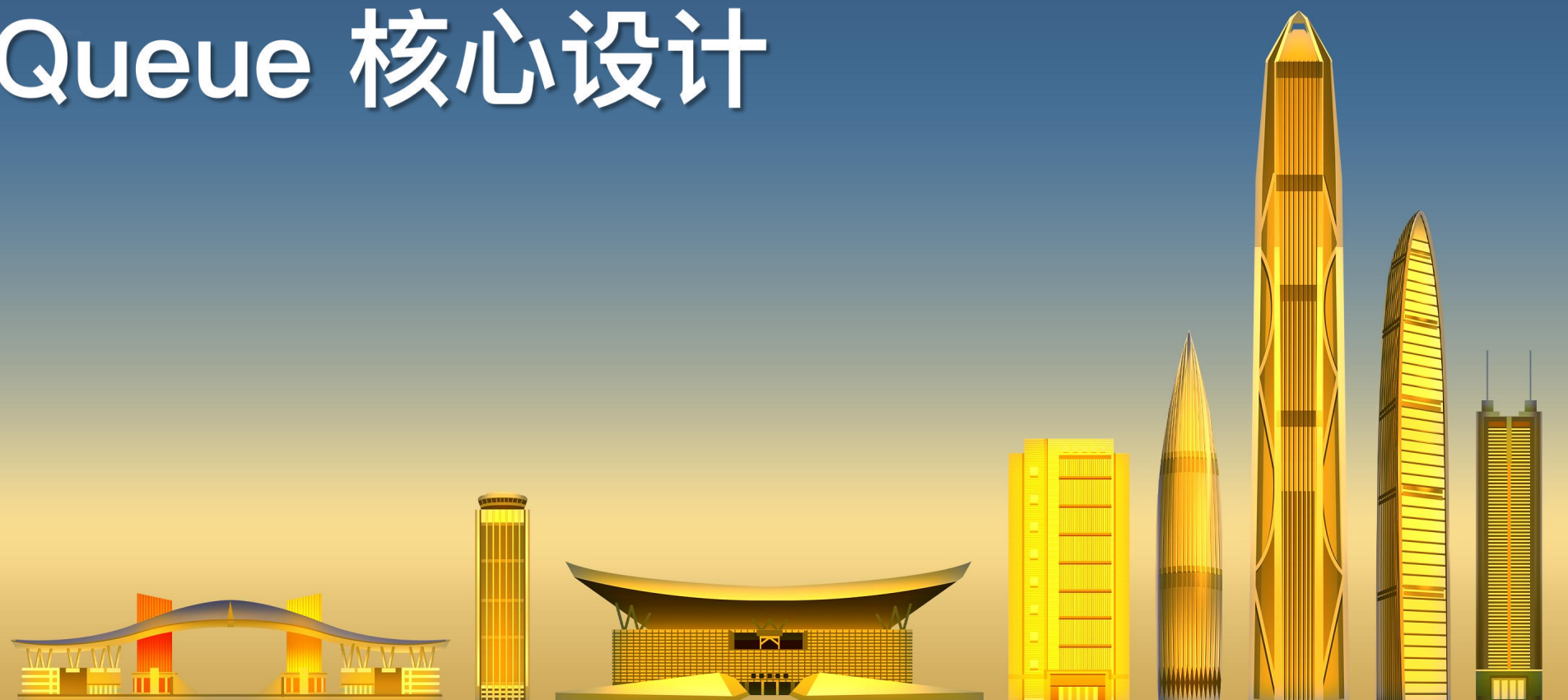
bounded reuse：

6,524 → 11,537 tokens/s（1.77×）

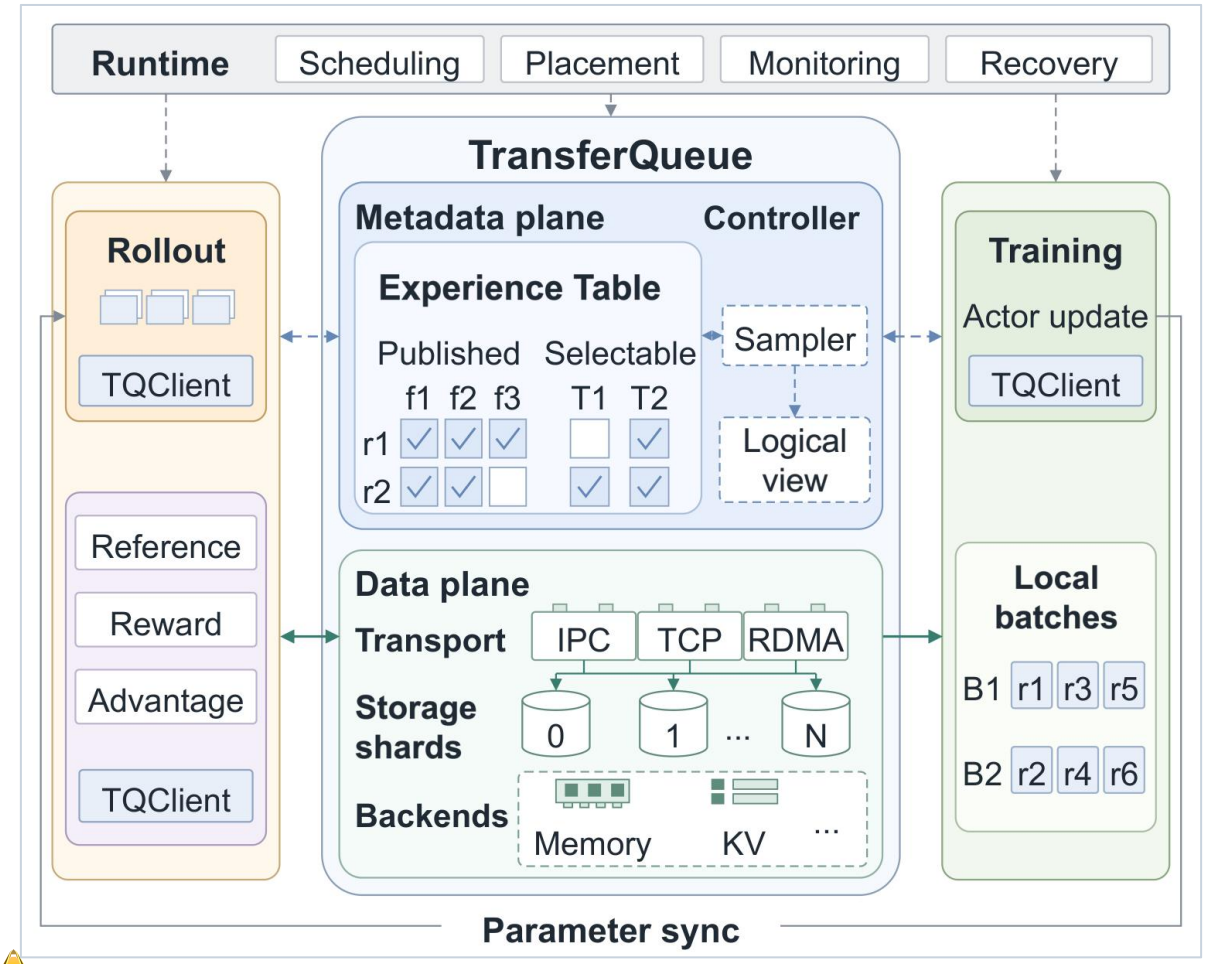
TransferQueue：管理共享的样本生产消费状态，链接RL框架和数据存储，实现数控分离

02

TransferQueue 核心设计



总体架构



① 活跃经验表

行 = 存活的经验记录，列 = 命名字段；表中只记录字段是否 Published、对各任务是否 Selectable，字段值留在分布式存储

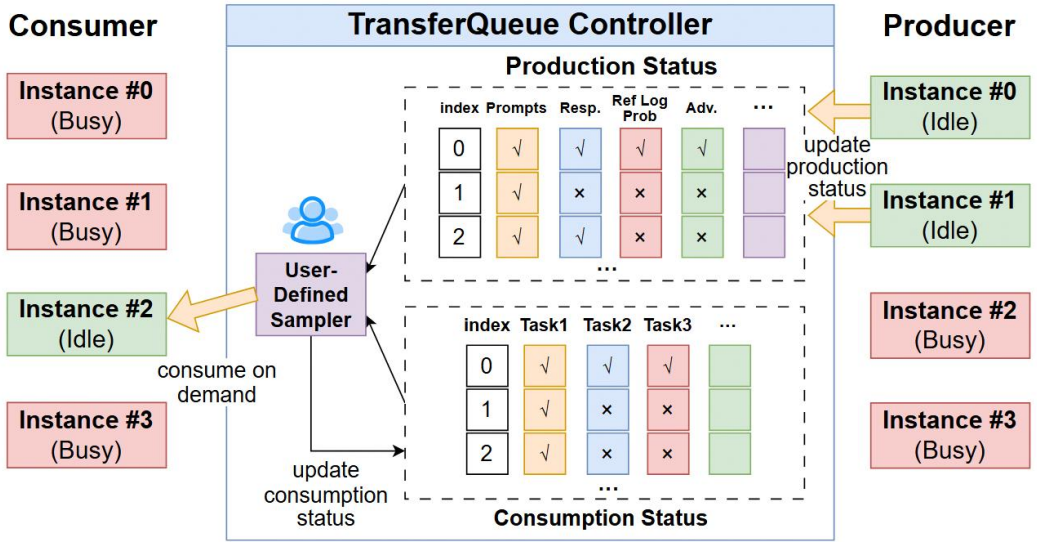
② 元数据面（Controller）

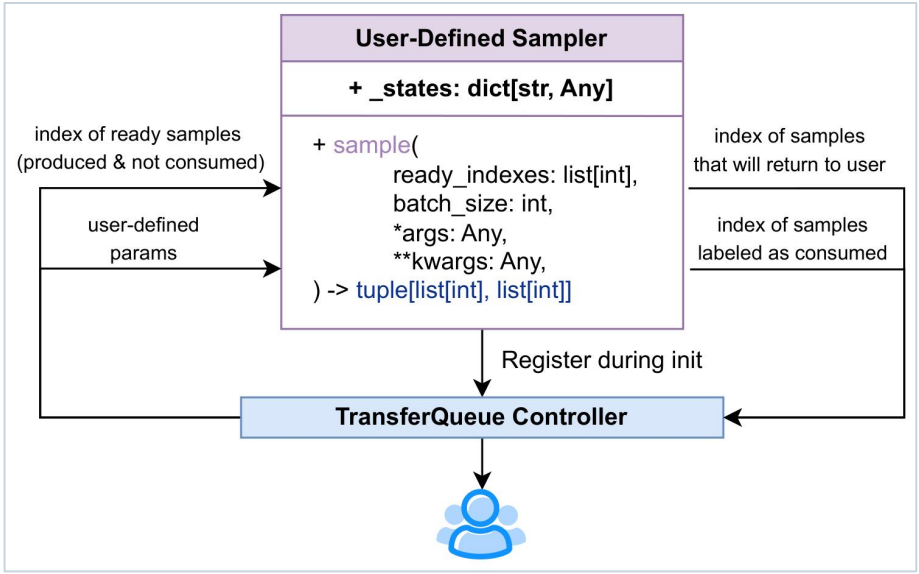
生产者写完字段即发布；任务按需请求字段，Controller 原子地选出就绪记录，并更新该任务的可选状态

③ 数据面（Storage）

字段分片存放在可插拔后端；Worker 根据meta信息**直接读写**，大张量数据不经过 Controller

元数据面：生产/消费矩阵 + 可插拔 Sampler





生产状态矩阵

记录 × 字段；字段写完即标记为已发布，不必等整条记录或整批完成

消费状态矩阵

记录 × 任务；跟踪各任务是否仍可选。**数据分区**隔离 train/val 与多策略版本，支持陈旧度控制

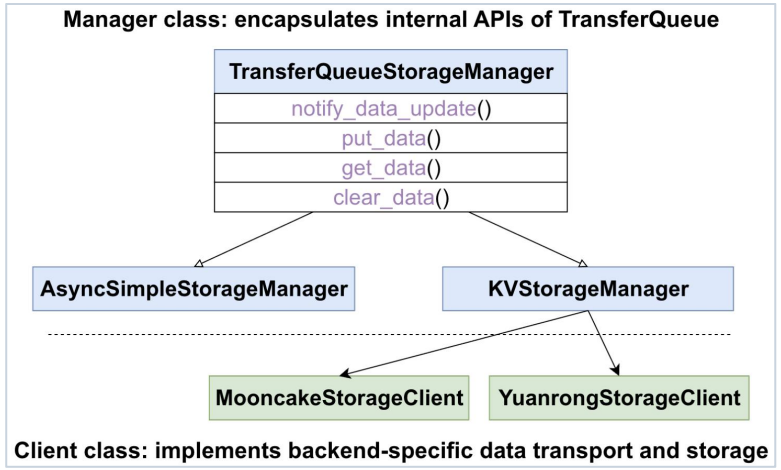
原子发放

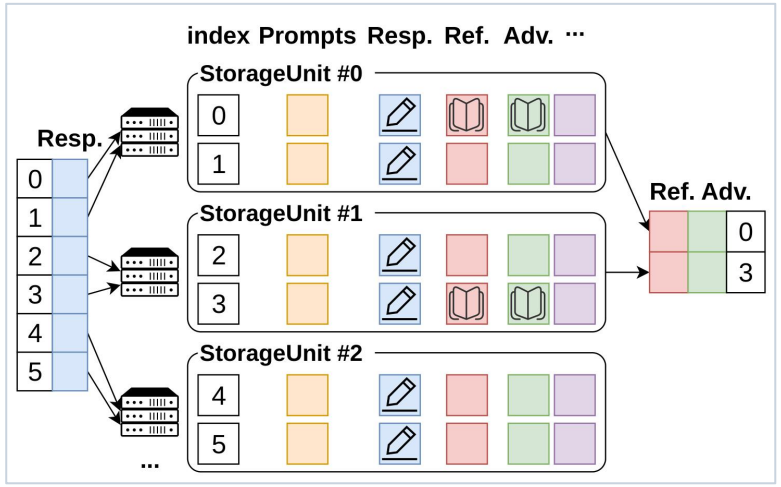
单个 Controller 串行化发放：选取操作、Sampler 状态与可选状态更新均为原子化操作；
分组请求需组内全员字段就绪

可插拔 Sampler

sample() 返回发放给调用者的记录，标记为已消费的记录，可表达一次性消费、bounded reuse 与回放

数据面：可插拔存储与字段级直读





后端	特点
SimpleStorage	主机内存 + ZMQ，零外部依赖，便于快速部署
Yuanrong	昇腾原生数据系统，HBM / DRAM / SSD 分层存储
MooncakeStore	KV 分层存储，支持 GPU 与 DRAM 间 RDMA 传输
Ray RDT	Ray Direct Transport：Ray actor 间直接存取对象

扩展新后端

继承 TransferQueueStorageManager（KV 类后端可基于 KVStorageManager），实现 put_data / get_data / clear_data

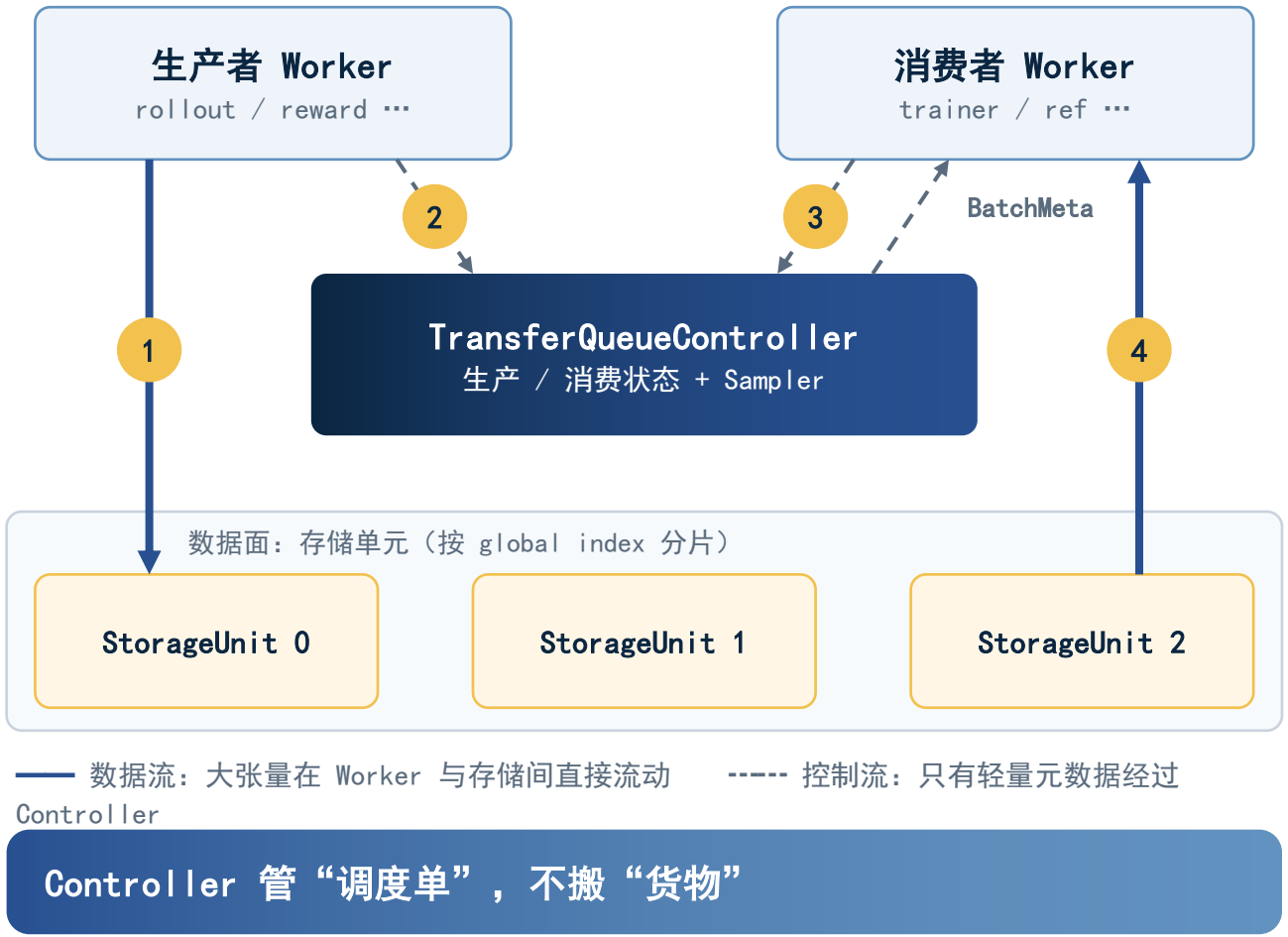
上层框架代码无需修改

字段级直读

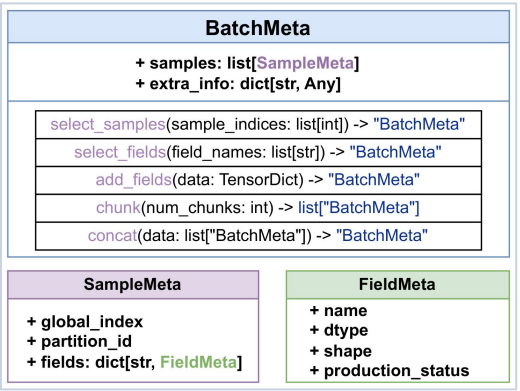
SimpleStorage 二维列式布局：行 = 记录（按 global index 分片），列 = 字段

元数据调度与数据 I/O 并发执行，形成流水

一条样本的旅程：数据流与控制流分离



1. put()：生产者把字段写入存储单元
2. 客户端通知 Controller：字段已发布
3. get_meta()：消费者声明字段与 batch_size，Sampler 原子选取并返回 BatchMeta
4. get_data()：按 BatchMeta 从存储单元直读所需字段
5. clear_samples()：用完释放；无任务再可选时回收



三层元数据

BatchMeta → SampleMeta → FieldMeta

只是描述符：不拥有记录生命周期，也不定义执行批次

支持 select_fields / chunk / concat，兼容单控制器分发

三层编程接口

KV 接口

高层 Put / Get / List / Clear

细粒度 √ 流式 ○ Sampler ✗

外部 ReplayBuffer / 单控制器负责分发；Redis 风格，上手最快

StreamingDataLoader

高层 PyTorch DataLoader

细粒度 √ 流式 √ Sampler √

训练侧按 micro-batch 流式取数，可预取

TransferQueueClient

底层 基于元数据

细粒度 √ 流式 √ Sampler √

put / get_meta / get_data / clear_samples，完全控制

```
import transfer_queue as tq
from transfer_queue import StreamingDataset, StreamingDataLoader

tq.init()                    # 每个进程初始化一次
dataset = StreamingDataset(
    config=config,           # 取自 TransferQueueController
    batch_size=2, micro_batch_size=2,
    data_fields=["input_ids", "advantages"], # 只取所需字段
    partition_id="train", task_name="update_task",
    dp_rank=dp_rank,
)
loader = StreamingDataLoader(dataset=dataset,
                             num_workers=2, prefetch_factor=2)
for batch, batch_meta in loader:
    ... # 与 PyTorch DataLoader 相同的训练循环
```

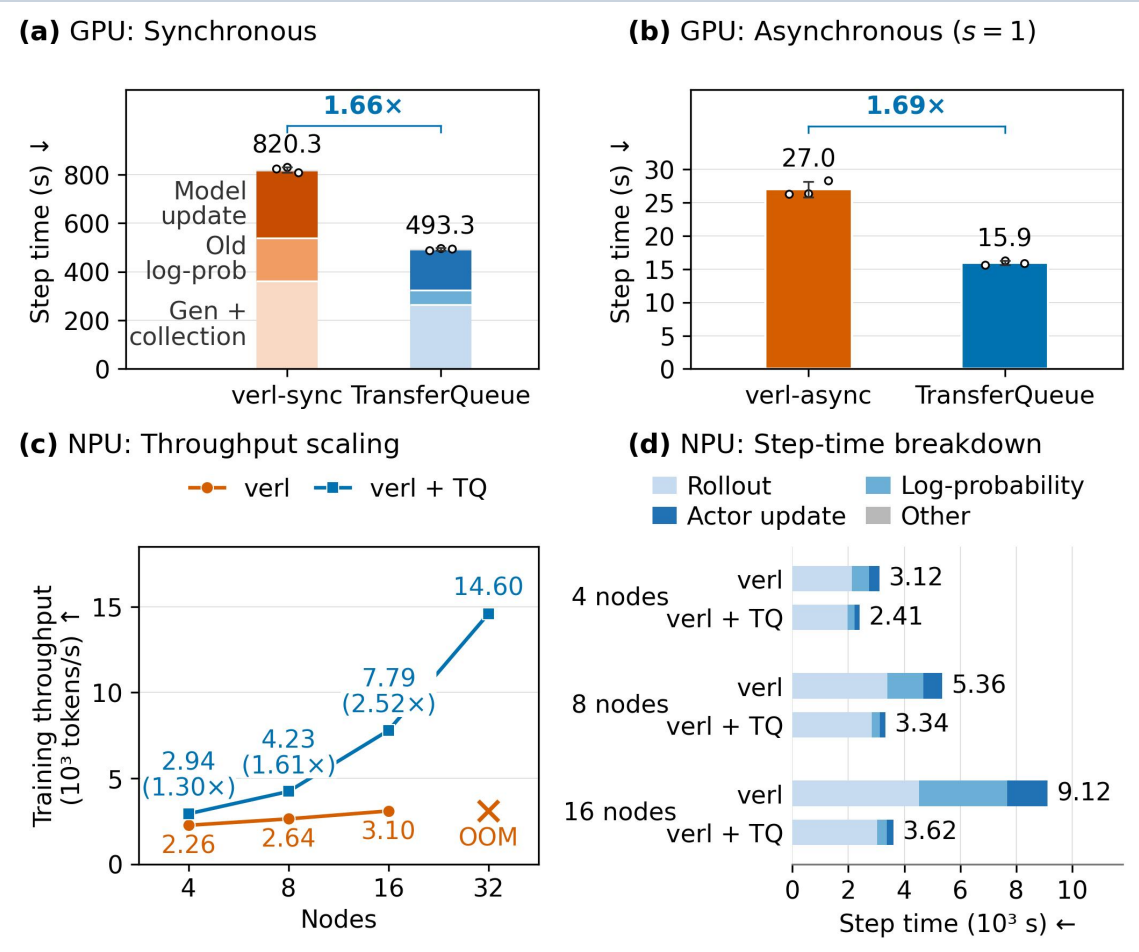
按需选层：从 KV 起步，需要流式与 Sampler 时下沉到 StreamingDataLoader / Client

03

实验结果



端到端：GPU 最高 1.69×，NPU 最高 2.52×



1.66×

H200 同步
Qwen2.5-VL-7B
820.3 → 493.3 s/step

1.69×

H200 异步（s = 1）
Qwen3-VL-8B
27.0 → 15.9 s/step

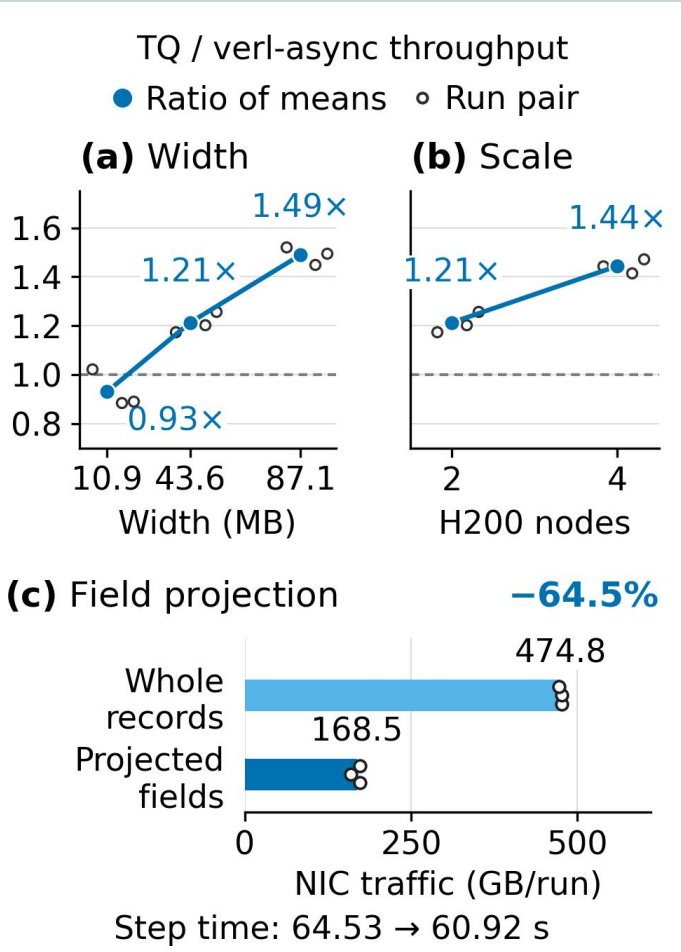
2.52×

Atlas A3 · 16 节点
256 NPU
训练吞吐

规模越大，收益越大（Atlas A3，同步 GRPO，Qwen3-VL-30B）

- 4/8/16节点：吞吐优势 1.30× → 1.61× → 2.52×
- 32节点512 NPU：TQ 达14600 tokens/s，非TQ下内存不足
- 4→16节点，log-prob + actor update：非TQ下 985.8 → 4589.9 s，TQ 仅 426.5 → 580.3 s
- 16节点时rollout约占 TQ step 的 84%，成为主要剩余瓶颈

数据面：记录越宽、规模越大，收益越高



1.49×

(a) Record大小：10.9 / 43.6 / 87.1 MB

相对verl-async 吞吐：0.93× / 1.21× / 1.49×

Record数据越大，集中调度的代价越高

1.44×

(b) 部署规模：4 / 8 / 16 节点

吞吐优势 1.30× → 1.61× → 2.52×

−64.5%

(c) 只读所需字段

NIC 流量 474.8 → 168.5 GB/step

收益随记录数据量大小和部署规模增长；只读所需字段进一步省掉无用流量

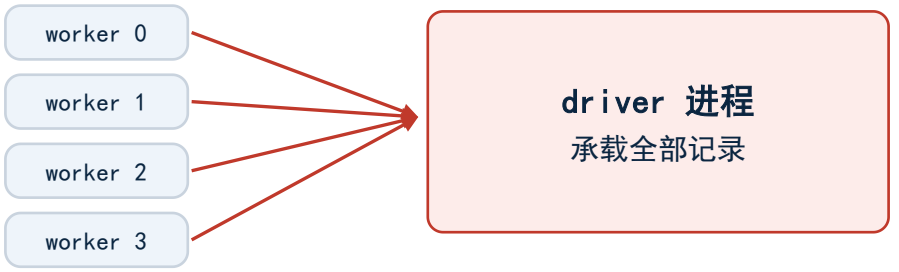
内存：从全局汇聚到按分片承载

场景	测量对象	非TQ	TQ
训练（22.3 GB 活跃记录）	driver 进程	96.5 GB	1.2 GB
训练（22.3 GB 活跃记录）	最大数据路径的进程	323 GB	53 GB
压测（529.9 GB 存储记录）	最大数据路径的进程	1.06 TB	33.8 GB

压测中 TQ 取 16 个 storage worker 的最大值；存储量翻倍到1.06TB时，最大worker为67.0GB

两次测量都接近全局载荷的1/16：内存随分片数摊薄，而不是压在一个进程上

集中式：全部记录汇聚到 driver

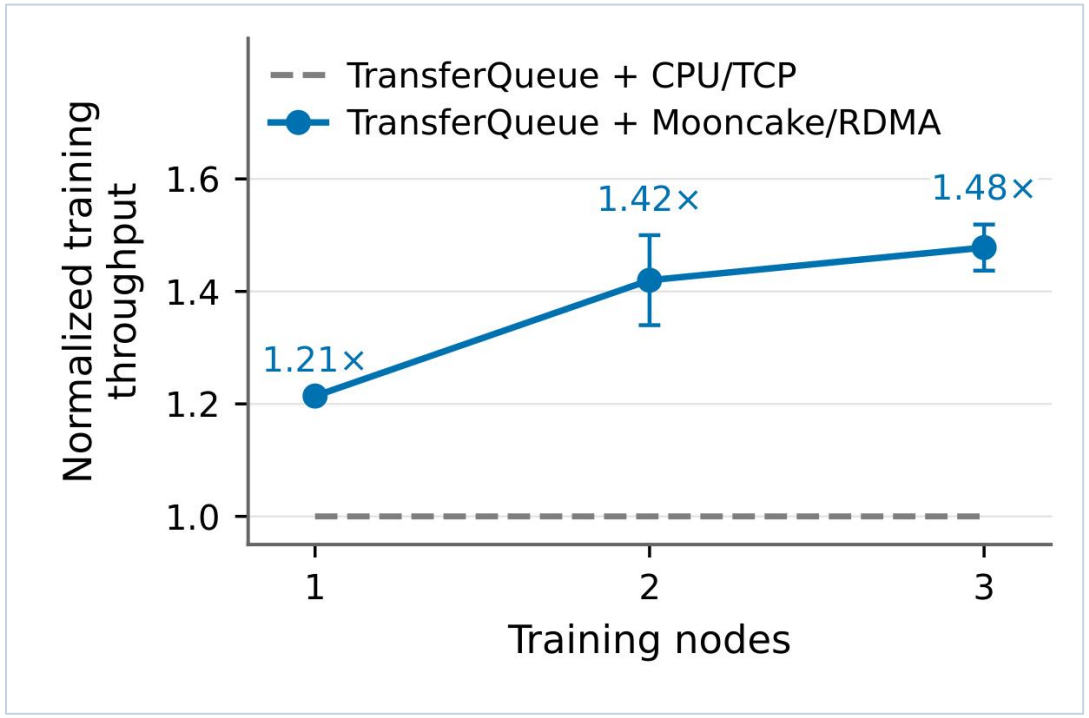


TQ：按 global index 分片

1/16	1/16	1/16	1/16
1/16	1/16	1/16	1/16
1/16	1/16	1/16	1/16
1/16	1/16	1/16	1/16

从“一个进程装下所有记录”到“每个 worker 只装自己那一份”

接口模块化：换后端、改策略无需改动训练代码



蒸馏：Qwen3-8B 教师 → Qwen3-0.6B 学生，65 MB logit 记录；1 个存储节点，训练节点 1 → 3

换后端：只改配置

- CPU/TCP → Mooncake/RDMA，record选择与训练代码不变
- 1 / 2 / 3 训练节点：1.21× / 1.42× / 1.48×

改策略：更换Sampler

verl　+273 / −40 行

改动 5 个模块

TQ　91 行 + 26 / −4 行

framework 侧 sampler + 集成；trainer、加速引擎不变

记录接口把“选择策略”与“载荷路径”隔开：各自演进，互不侵入

04 开源项目与生态



开源项目：TransferQueue

TransferQueue

开源协议	Apache-2.0 · Python ≥ 3.10
安装	pip install TransferQueue
可选后端	pip install TransferQueue[mooncake] / [yuanrong]
教程	01 核心组件 → 02 KV 接口 → 03 元数据 → 04 Controller → 05 自定义 Sampler → 06 StreamingDataLoader
示例	recipe/simple_use_case：single_controller_demo、relax_demo

压测：8192 个并发客户端、4 节点写入 2 TB，无崩溃、无数据丢失

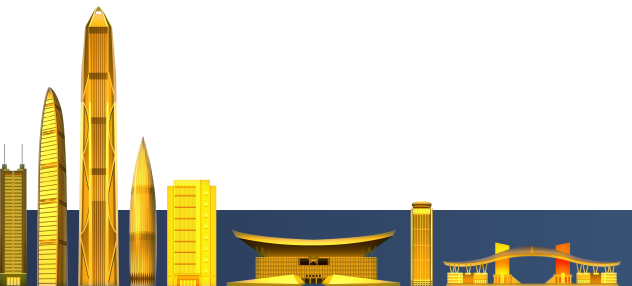
Quickstart：Redis 风格 KV 接口

```
import torch, transfer_queue as tq
from tensordict import TensorDict

tq.init()
batch = TensorDict({"input_ids": torch.tensor(1, 2, 3], [4, 5, 6),
                    "attention_mask": torch.ones(2, 3)}, batch_size=2)
tq.kv_batch_put(keys=["1_0", "1_1"], partition_id="Train",
                fields=batch, tags=[{"model_version": 1}] * 2)

# 字段级读取：只取 input_ids
out = tq.kv_batch_get(keys=["1_0", "1_1"], partition_id="Train",
                      select_fields="input_ids")
print(tq.kv_list(partition_id="Train"))
tq.kv_clear(keys=["1_0", "1_1"], partition_id="Train")
```

https://github.com/Ascend/TransferQueue · 欢迎 Issue / PR



生态连接：上接框架，下接后端

后训练框架

- ByteDance Seed — verl: Volcano Engine Reinforcement Learning for LLMs — 字节Verl
- ROLL — like a Reinforcement Learning Algorithm Developer — 阿里Roll
- Relax — 小红书Relax
- UniRL — 腾讯混元UniRL
- Meshy — OpenBMB
- Accio Dressage — Accio-Lab

TransferQueue

元数据面
就绪 · 选取 · 复用

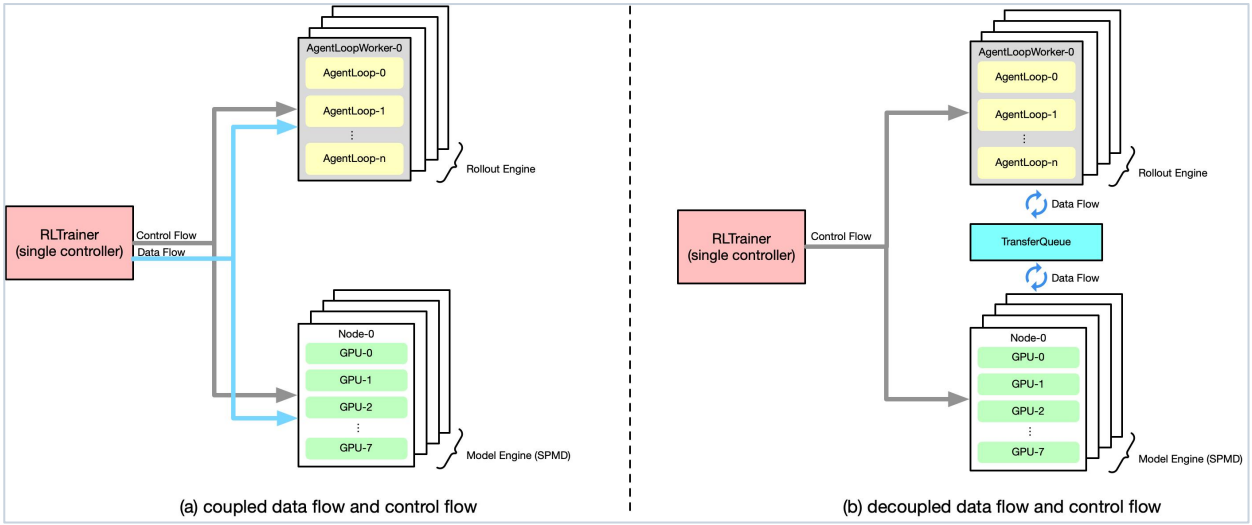
数据面
分片存储 · 字段直读

存储与传输后端

- OpenYuanrong — Ascend 原生，HBM / DRAM / SSD 分层
- Mooncake — KV 分层存储，GPU↔DRAM RDMA
- ØMQ — SimpleStorage 默认传输
- Ray RDT — Ray RDT：actor 间直传

框架可只用数据面、保留自己的样本选择逻辑，也可同时用元数据面负责样本发放

实践 verl × TQ：数控分离



左：数控耦合，大张量经中心 RLTrainer 汇聚转发；右：控制器只传元数据与引用，数据经 TQ 数据面流动

−49.2%

端到端耗时 756 → 384 s

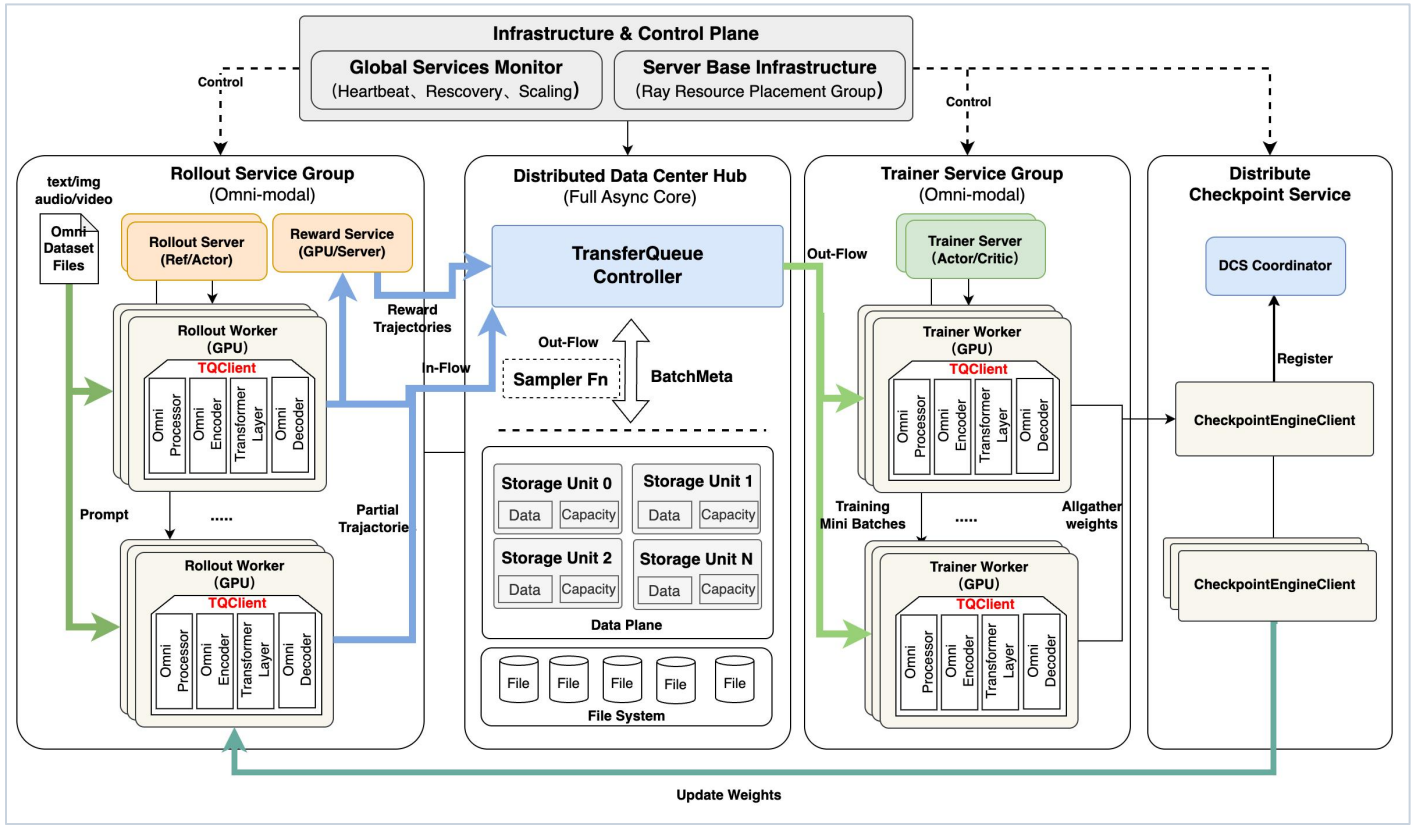
128×H100 · Qwen3-VL-30B-A3B · OneThinker · 16K/2K · GBS 128 · n = 8

按相同工作量折算速率约 1.97×

平台 / 规模	模型	数据集	长度	收益
H100 2×8	Qwen3-30B-A3B	DAPO-Math	2K/8K	+14.1%
H100 4×8	Qwen3-VL-30B-A3B	Geo3K	1K/2K	+12.5%
H100 2×8	Qwen3-VL-30B-A3B	OneThinker	16K/2K	+22.4%
H100 16×8	**Qwen3-VL-30B-A3B**	**OneThinker**	**16K/2K**	**+49.2%**
A3 2×16	Qwen3-VL-30B-A3B	Geo3K	1K/1K	+7%
A3 2×16	Qwen3-VL-30B-A3B	OneThinker	16K/1K	+20%

保留单控制器的编排便利优势的同时，把最重的数据路径移出控制器

实践 Relax × TQ：流式数据总线



去掉长尾的整批屏障

Controller + 自定义 Sampler 显式表达消费条件：哪些样本就绪、怎样组批、哪些版本可接受

micro-batch 粒度流式读取与预取

训练侧 StreamingDataLoader 按 micro-batch 读取与预取，让数据准备与前一段计算重叠

Stateness控制

异步训练通过staleness来控制旧轨迹

总结

1 问题	后训练规模扩大后，瓶颈从计算转向数据如何组织、传递与消费；根因是经验 - 批次所有权错配
2 方法	Field Readiness：元数据面管理样本的就绪和字段的可选性，数据面分片存储和读取；支持可插拔的存储后端和样本调度sampler
3 效果	GPU 1.66× / 1.69×，256 NPU 2.52×；已接入 verl、ROLL、Relax、UniRL 等框架



GOSIM

GOSIM Shenzhen 2026

Thanks!

